

Python Terminal Bank Simulator

A simple **Python-based Terminal Bank Simulator** that demonstrates basic banking operations, savings interest calculations, and investment simulations using Object-Oriented Programming (OOP).

This project is designed as a beginner-friendly Python project for understanding classes, objects, functions, user input, calculations, dictionaries, exception handling, and basic program flow.

Project Overview

The **Python Terminal Bank Simulator** is a command-line banking application where users can:

- Check their current bank balance
- Calculate savings growth with compound interest
- Explore different investment options
- Simulate investment returns
- Cash out simulated investments
- Exit the banking system

The program starts with a default balance of **\$1,000,000.00** and uses a **7%** annual savings interest rate.

> **Note:** This is an educational simulation only. It does not represent a real banking or investment system.

Features

1. Check Balance

Displays:

- Account owner's name
- Current account balance

Example:

```text

=====

Account Owner: Yuvraj

Current Balance: \$1,000,000.00

=====

---

## 2. Savings Interest Calculator

Users can enter the number of years for which they want to calculate savings growth.

The program uses the compound interest formula:

$$A = P(1 + r)^t$$

Where:

- P = Principal amount
- r = Annual interest rate
- t = Number of years
- A = Future balance

The default annual savings interest rate is:

7%

The program displays:

- Total interest earned
- Projected future balance

---

### 3. Investment Marketplace

The program provides three simulated investment options:

| Option | Investment        | Estimated Annual Return |
|--------|-------------------|-------------------------|
| 1      | High-Yield Stocks | 10%                     |
| 2      | Real Estate Trust | 7%                      |
| 3      | Cryptocurrency    | 30%                     |

The user can:

1. Select an investment
2. Enter the investment amount
3. Select the holding period
4. Calculate the final investment value
5. View the simulated profit/loss
6. Choose whether to cash out

---

### Investment Calculation

The investment value is calculated using compound growth:

Final Value = Investment  $\times (1 + \text{Rate})^{\text{Years}}$

The profit is calculated as:

Profit = Final Value - Initial Investment

For example, if \$10,000 is invested at an estimated return of 10% for 2 years:

Final Value =  $10000 \times (1.10)^2$

= \$12,100

Therefore:

Profit = \$12,100 - \$10,000

= \$2,100

---

## Technologies Used

- **Python 3**
  - Object-Oriented Programming (OOP)
  - Classes and Objects
  - Functions and Methods
  - Dictionaries
  - User Input
  - Exception Handling
  - Mathematical Calculations
  - time module
- 

## Project Structure

Python-Terminal-Bank-Simulator/

|

├── bank\_simulator.py

└── README.md

### **bank\_simulator.py**

Contains the complete Python source code for the banking simulator.

### **README.md**

Contains the project documentation and instructions.

---

## How to Run

### Step 1: Install Python

Make sure Python 3 is installed on your computer.

Check your Python version:

```
python --version
```

or:

```
python3 --version
```

---

### **Step 2: Clone the Repository**

```
git clone https://github.com/your-username/Python-Terminal-Bank-Simulator.git
```

---

### **Step 3: Open the Project Folder**

```
cd Python-Terminal-Bank-Simulator
```

---

### **Step 4: Run the Program**

```
python bank_simulator.py
```

---

### **Program Flow**

When the program starts, it asks for the user's name:

Welcome to the Python Terminal Bank Simulator!

Please enter your name: Yuvraj

The main menu is then displayed:

--- MAIN MENU ---

1. Check Balance
2. Calculate Savings Interest
3. Explore Investments
4. Exit

What would you like to do? (1-4):

The user can select any available operation.

---

### **Input Validation**

The program includes basic input validation and error handling.

For example:

- Negative years are rejected.
- Invalid numbers are handled using try-except.
- Investment amounts greater than the available balance are rejected.
- Zero or negative investment amounts are rejected.
- Invalid menu choices display an error message.

Example:

Invalid option. Please choose 1-4.

---

### **Object-Oriented Programming**

The project uses a class called:

class BankSimulator:

The class contains:

#### **Constructor**

`__init__()`

Used to initialize:

- Account owner
- Account balance
- Savings interest rate

#### **Methods**

The class contains the following methods:

`check_balance()`

`calculate_savings()`

`invest_options()`

This structure demonstrates the use of **Object-Oriented Programming in Python**.

---

### **Important Disclaimer**

This project is created **for educational and demonstration purposes only**.

The investment returns shown in the program are simulated values and are **not guaranteed real-world returns**.

The project does not connect to:

- Real bank accounts
- Real payment systems
- Stock markets
- Cryptocurrency exchanges
- Financial institutions

No real money is transferred or invested.

---

### **Future Improvements**

Possible improvements for future versions include:

- Add user login and password authentication
- Add deposit and withdrawal functionality
- Add transaction history
- Add multiple bank accounts
- Store account data using files or a database
- Add an account number
- Add PIN verification
- Add monthly transaction statements
- Add graphical user interface (GUI)
- Add real-time market data
- Add persistent investment portfolios
- Add database support using SQLite or MySQL

---

## Learning Objectives

This project helps demonstrate the following Python concepts:

- Variables and data types
- Input and output
- Conditional statements
- Loops
- Functions
- Classes and objects
- Dictionaries
- Exception handling
- Mathematical operations
- String formatting
- The time module
- Basic financial calculations

---

## License

This project is intended for educational purposes.

You are free to use, modify, and improve the code for learning and academic projects.

---

## Author

**Yuvraj Suman**

B.Tech Computer Science Engineering

---

If you find this project useful, consider giving the repository a **star** on GitHub!

### Recommended GitHub repository structure

Rename your Python file to:

```
```text
```

bank_simulator.py

and keep the repository like this:

Python-Terminal-Bank-Simulator

|

└─ bank_simulator.py

└─ README.md

Then GitHub will automatically display README.md on the main repository page.